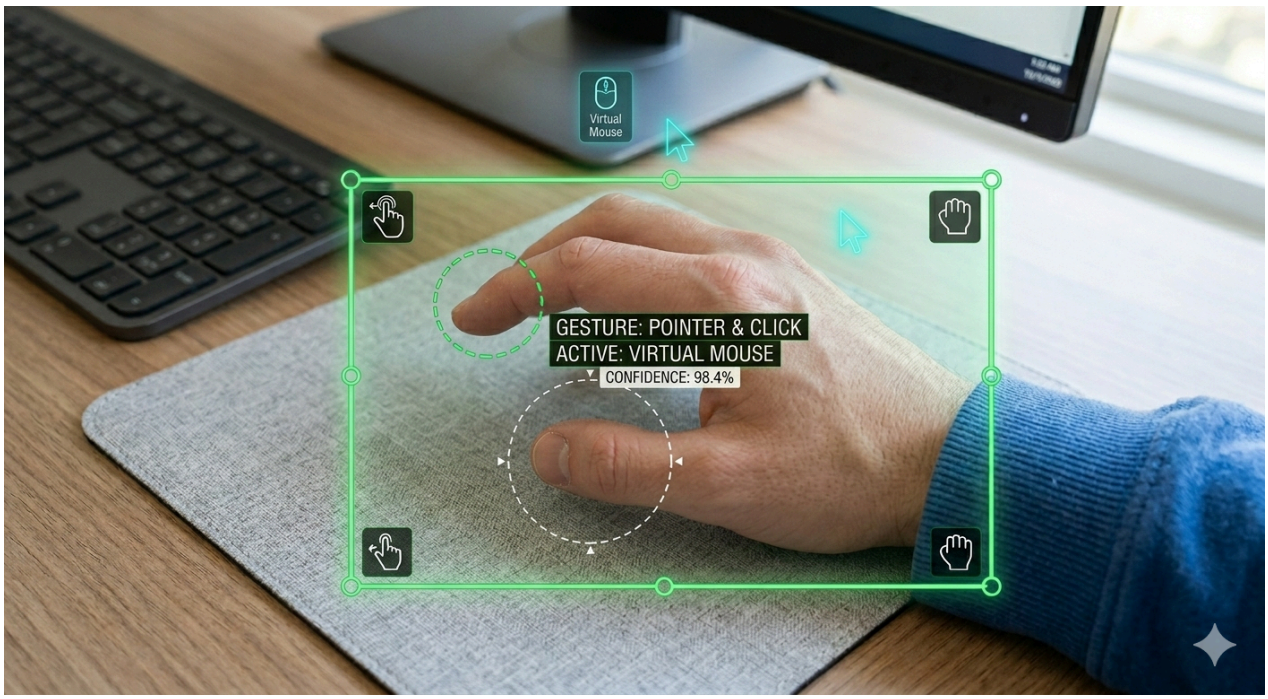
Mouse Virtual dengan Pengenalan Gerakan Tangan

Notebook ini mendemonstrasikan aplikasi mouse virtual yang mengontrol kursor komputer dan melakukan tindakan klik/scroll berdasarkan gerakan tangan yang terdeteksi melalui webcam. Ini memanfaatkan `mediapipe` untuk deteksi landmark tangan dan `pyautogui` untuk mengontrol mouse.



1. Impor dan Konfigurasi

Bagian ini mengimpor pustaka yang diperlukan dan mendefinisikan parameter konfigurasi utama untuk aplikasi, seperti jalur model Mediapipe, resolusi layar, dan parameter penyetelan untuk pergerakan mouse dan scrolling.

```
import cv2
import pyautogui
import time
import mediapipe as mp
from mediapipe.tasks import python
from mediapipe.tasks.python import vision

# ================= CONFIG =================

# Path ke model Hand Landmarker yang sudah dilatih. File ini harus diunduh secara terp
MODEL_PATH = r"C:\codingan\project\hand_landmarker.task"

# Resolusi layar untuk memetakan gerakan tangan ke posisi kursor
SCREEN_W, SCREEN_H = 1920, 1200

# SMOOTHENING didefinisikan tetapi tidak digunakan dalam kode yang disediakan. Ini mun
SMOOTHENING = 3
# Penundaan minimum antara klik berturut-turut untuk mencegah klik yang tidak disengaj
```

```
CLICK_DELAY = 0.25
# Jumlah frame yang dilewati untuk meningkatkan kinerja dan berpotensi memperlancar ge
FRAME_SKIP = 2

# ===== Penyetelan Scroll ( 🔥 ini kunci mulus) =====
# Faktor peredam untuk kecepatan scroll, membuat scrolling lebih halus
SCROLL_DAMPING = 0.85
# Faktor penguatan untuk menyesuaikan kecepatan scroll berdasarkan gerakan tangan
SCROLL_GAIN = 0.04
# Gerakan tangan vertikal minimum untuk memulai scrolling
SCROLL_DEADZONE = 2
# Gerakan tangan horizontal/vertikal minimum untuk mendaftarkan gerakan kursor, menceg
DEADZONE = 5          # anti goyang kecil tangan
# Filter Alpha untuk menghaluskan gerakan kursor. Nilai yang lebih tinggi berarti lebi
FILTER_ALPHA = 0.8   # 0.5–0.8 → makin besar makin halus
# ==========================================

# Variabel untuk menyimpan posisi kursor sebelumnya dan waktu klik terakhir
prev_x, prev_y = 0, 0
last_click_time = 0
frame_count = 0

# Variabel untuk scrolling yang mulus
scroll_prev_y = None
scroll_velocity = 0.0
```

2. Pengaturan Hand Landmarker

Bagian ini menginisialisasi MediaPipe Hand Landmarker, yang bertanggung jawab untuk mendeteksi landmark tangan dalam frame video. Ini menentukan model yang akan digunakan dan mengkonfigurasinya untuk pemrosesan video.

```
# ================ LANDMARKER ================

# Impor kelas yang diperlukan dari MediaPipe Tasks
BaseOptions = python.BaseOptions
HandLandmarker = vision.HandLandmarker
HandLandmarkerOptions = vision.HandLandmarkerOptions
VisionRunningMode = vision.RunningMode

# Konfigurasi Hand Landmarker dengan model yang sudah dilatih dan mode berjalan video
options = HandLandmarkerOptions(
    base_options=BaseOptions(model_asset_path=MODEL_PATH),
    running_mode=VisionRunningMode.VIDEO,
    num_hands=1 # Mendeteksi satu tangan
)

# Buat instance Hand Landmarker
landmarker = HandLandmarker.create_from_options(options)
```

3. Pengaturan Kamera

Bagian ini mengatur webcam untuk pengambilan video, mendefinisikan resolusi untuk frame input.

```
# ================ CAMERA ================

# Inisialisasi pengambilan video dari webcam default (indeks 0)
cap = cv2.VideoCapture(0)
# Atur lebar dan tinggi frame
cap.set(3, 320)
cap.set(4, 240)
```

4. Fungsi Deteksi Keadaan Jari

Fungsi utilitas ini menganalisis landmark tangan yang terdeteksi untuk menentukan keadaan setiap jari (apakah jari tersebut lurus atau ditekuk). Ini sangat penting untuk pengenalan gerakan.

```
# ================ FINGER STATES ================

# Fungsi untuk menentukan apakah jari lurus atau ditekuk
def finger_states(lm):
    return {
        "index":  lm[8].y  < lm[6].y,  # Ujung jari telunjuk (8) berada di atas buku j
        "middle": lm[12].y < lm[10].y, # Ujung jari tengah (12) berada di atas buku ja
        "ring":   lm[16].y < lm[14].y, # Ujung jari manis (16) berada di atas buku jar
        "pinky":  lm[20].y < lm[18].y, # Ujung jari kelingking (20) berada di atas buk
        "thumb":  lm[4].x  > lm[3].x   # Ujung jempol (4) berada di kanan pangkalnya (
    }
```

5. Loop Utama dan Pengenalan Gerakan

Loop `while True` terus-menerus menangkap frame dari webcam, memprosesnya dengan Hand Landmarker, dan menginterpretasikan gerakan tangan untuk mengontrol mouse.

Definisi Gerakan:

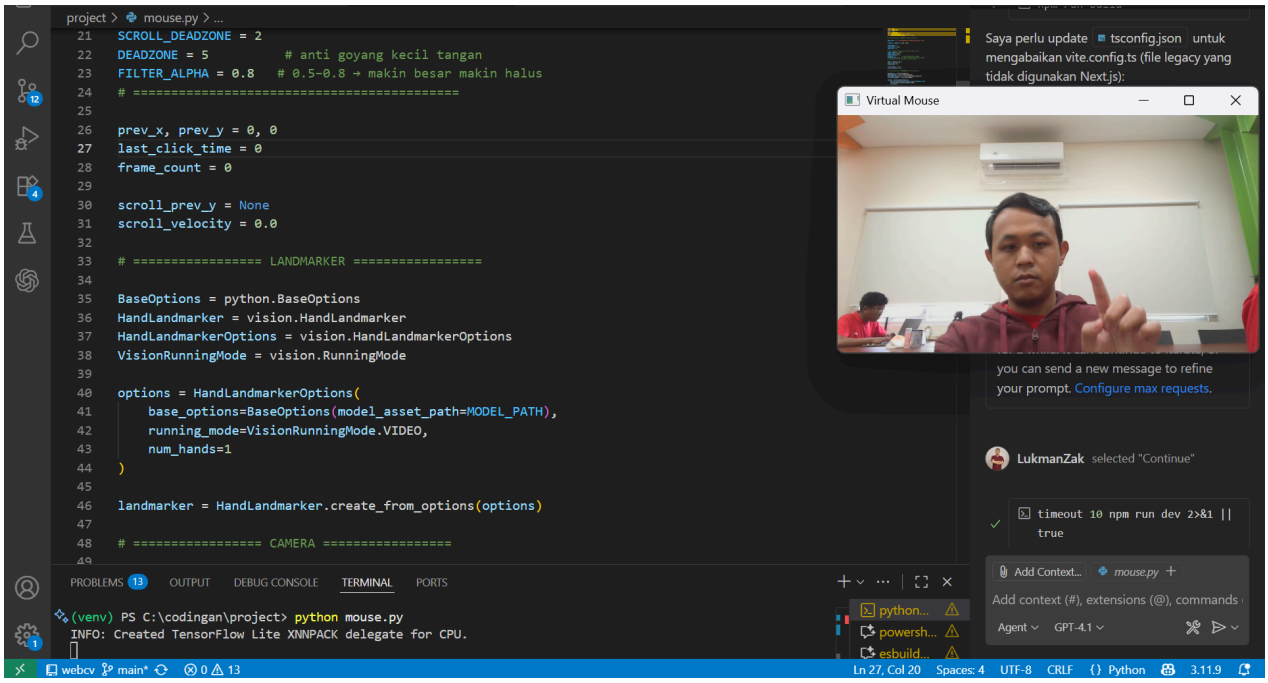
- **Gerakan Kursor**: Hanya jari telunjuk yang lurus.
- **Klik Kiri**: Jari telunjuk dan jari tengah lurus.
- **Klik Kanan**: Jari telunjuk, jari tengah, dan jari manis lurus.
- **Scroll**: Keempat jari (telunjuk, tengah, manis, kelingking) lurus.

```
# ================ MAIN LOOP ================

while True:
    # Baca frame dari kamera
    success, frame = cap.read()
    if not success:
        continue

    frame_count += 1
    # Balik frame secara horizontal untuk efek cermin alami
    frame = cv2.flip(frame, 1)

    # Lewati frame untuk meningkatkan kinerja dan menghaluskan gerakan
    if frame_count % FRAME_SKIP != 0:
        cv2.imshow("Virtual Mouse", frame)
        if cv2.waitKey(1) & 0xFF == 27: # Tekan 'Esc' untuk keluar
            break
        continue
```

```
        # Konversi frame ke format RGB yang diperlukan oleh MediaPipe
        rgb_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)

        # Buat objek MediaPipe Image
        mp_image = mp.Image(
            image_format=mp.ImageFormat.SRGB,
            data=rgb_frame
        )

        # Dapatkan stempel waktu saat ini dalam milidetik
        timestamp_ms = int(time.time() * 1000)
        # Deteksi landmark tangan dalam frame
        result = landmarker.detect_for_video(mp_image, timestamp_ms)

        if result.hand_landmarks:
            # Dapatkan landmark untuk tangan pertama yang terdeteksi
            lm = result.hand_landmarks[0]
            # Tentukan keadaan setiap jari
            states = finger_states(lm)

            index  = states["index"]
            middle = states["middle"]
            ring   = states["ring"]
            pinky  = states["pinky"]

            # Petakan koordinat ujung jari telunjuk ke koordinat layar
            x_screen = lm[8].x * SCREEN_W
            y_screen = lm[8].y * SCREEN_H

            # Hitung perpindahan dari posisi sebelumnya
            dx = x_screen - prev_x
            dy = y_screen - prev_y

            # Terapkan deadzone untuk mencegah jitter kecil
            if abs(dx) < DEADZONE:
                dx = 0
            if abs(dy) < DEADZONE:
                dy = 0

            # Terapkan filter untuk gerakan kursor yang lebih halus
            curr_x = prev_x + dx * FILTER_ALPHA
            curr_y = prev_y + dy * FILTER_ALPHA

            now = time.time()

            # ================= GESTURES =================

            # ✅ GERAK → hanya jari telunjuk yang lurus
            if index and not middle and not ring and not pinky:
                pyautogui.moveTo(curr_x, curr_y, _pause=False)
                scroll_prev_y = None # Reset status scroll
                scroll_velocity = 0

            # ✅ KLIK KIRI → jari telunjuk + jari tengah lurus
            elif index and middle and not ring and not pinky:
                if now - last_click_time > CLICK_DELAY: # Mencegah beberapa klik dalam int
                    pyautogui.click()
                    last_click_time = now

                scroll_prev_y = None # Reset status scroll
```

```
                scroll_velocity = 0

            # ✅ KLIK KANAN → tiga jari lurus
            elif index and middle and ring and not pinky:
                if now - last_click_time > CLICK_DELAY:
                    pyautogui.rightClick()
                    last_click_time = now

                scroll_prev_y = None # Reset status scroll
                scroll_velocity = 0

            # ✅ SCROLL → empat jari lurus (mode mulus)
            elif index and middle and ring and pinky:

                if scroll_prev_y is None: # Inisialisasi scroll jika baru dimulai
                    scroll_prev_y = curr_y
                    scroll_velocity = 0
                else:
                    dy = curr_y - scroll_prev_y
                    scroll_prev_y = curr_y

                    if abs(dy) > SCROLL_DEADZONE: # Hanya scroll jika gerakan melebihi dea
                        # Terapkan peredam dan penguatan untuk scrolling mulus
                        scroll_velocity = (
                            scroll_velocity * SCROLL_DAMPING
                            + dy * SCROLL_GAIN * 100
                        )

                        pyautogui.scroll(int(-scroll_velocity)) # Gulir ke atas atau ke ba

            else:
                scroll_prev_y = None # Reset status scroll untuk gerakan lain
                scroll_velocity = 0

            # Perbarui koordinat sebelumnya untuk frame berikutnya
            prev_x, prev_y = curr_x, curr_y

        else:
            # Jika tidak ada tangan yang terdeteksi, reset status scroll
            scroll_prev_y = None
            scroll_velocity = 0

        # Tampilkan frame yang diproses
        cv2.imshow("Virtual Mouse", frame)

        # Hentikan loop jika 'Esc' ditekan
        if cv2.waitKey(1) & 0xFF == 27:
            break

    # Lepaskan kamera dan tutup semua jendela OpenCV setelah selesai
    cap.release()
    cv2.destroyAllWindows()
```

Contoh Hasil



Start coding or generate with AI.